

Collections Framework

18 July 2026 07:06

Array : Array are basic container, collection of similar data type

limitation of array

Array is fixed in nature

Array can store similar kind data type at once

Collections are growable in nature and can store both homogeneous and heterogeneous values.

In collection every class is implemented based on some standard class (data structure) it means every class will have some predefined capability.

```
List<Integer> list = new ArrayList();
```

Wrapper classes are important because java collection store object, not primitive that's is why we write List<Integer> not List<int>

If collection storing object then which this line is running : list.add(10);

Primitive Type Vs Wrapper Class

Primitive type store simple value, wrapper class store values as an object.

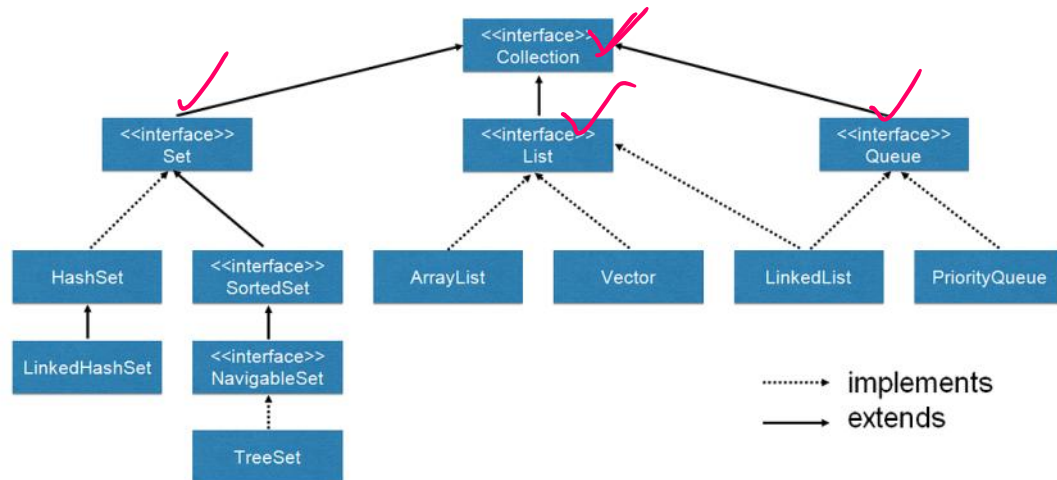
Primitive Type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Why do we need wrapper classes ?

Main reason:

1. Collections store objects, not primitive
2. Wrapper class provide useful methods
3. Wrapper object can store null
4. Wrapper classes are useful in generics

Collection Interface



What problem does the collection framework solve ?

Int m1=100;

Int m2=90;

Int m3=80;

Int m4=79;

We have array to resolve above problem.

If we have 100 student ? --> array

If we have 1000 student ? --> array

We need data structure that can:

1. Grow and shrink dynamically
2. Store multiple objects
3. Search efficiently
4. Remove duplicates
5. Maintain insertion order
6. Process data in FIFO or LIFO
7. Store key-value relationship

What is java collections framework ?

Java collections framework is group of:

1. Interface
2. Implementation classes
3. Algorithms

Collection Vs Collections Vs Collection Framework

1. Collection: is an interface

Collection<String> name = new ArrayList<>();

Common methods:

1. Add()

2. Remove()
3. Contains()
4. Size()
5. isEmpty()
6. Clear()
7. Iterator()

Collections

Is a utility class it contains static helper methods:

`Collections.sort(list)`

`Collection.reverse(list)`

Collection framework

Is complete architecture containing

1. Interfaces
2. Implementation classes
3. Algo
4. Iterators
5. Utility classes
6. Concurrent collections

Why do collection store object, not primitive ?

Collections use generics:

`Collection<Integer> name = new ArrayList<>();`

We cannot write:

`Collection<int> name = new ArrayList<>();` --> not allowed

Collection store object reference, therefore primitive values are converted into wrapper object

Note: in primitive we cannot do cloning of objects.

Why were array not enough ?

`Int[] nums=new int[5];`

Array limitation:

1. Fixed size
2. Array does not have built-in add or remove operations
3. Manual shifting may be required
4. Few high level operations
5. Difficult to manage dynamic data

Collection Advantage:

```
List<Integer> num = new ArrayList<>();
List<Integer> nums = new ArrayList<>();
nums.add(10);
nums.add(20);
nums.add(30);
```

```
nums.remove(Integer.valueOf(20));
System.out.println(nums);
```

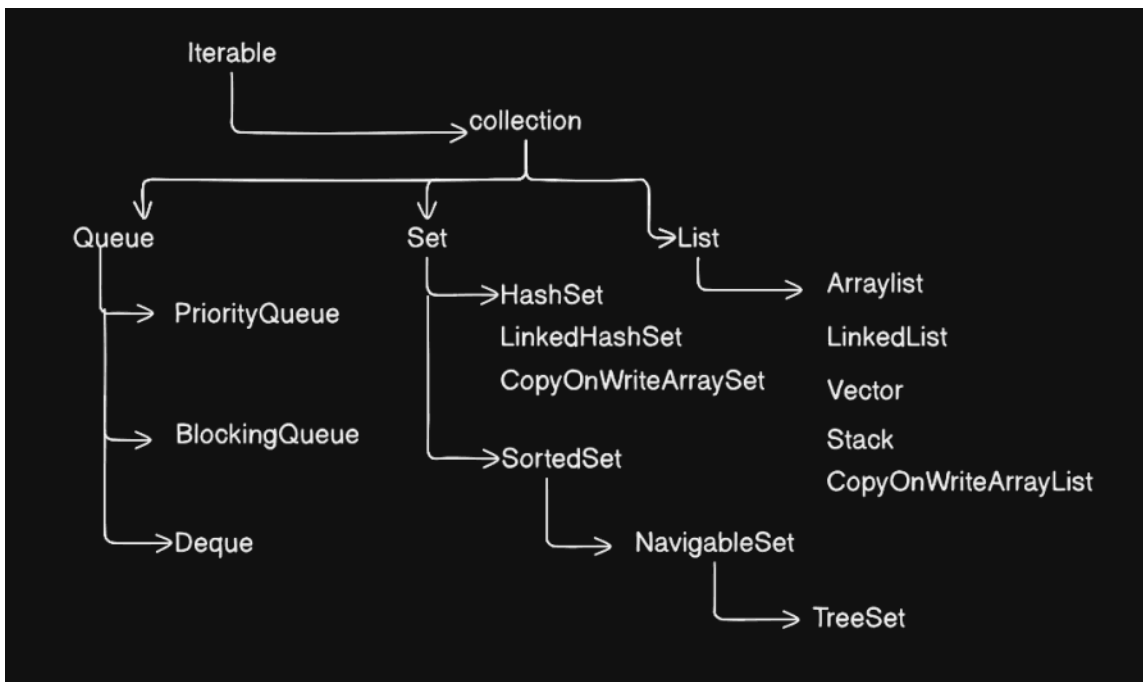
The collection manage resizing and shifting internally

Complete high-level hierarchy

Morden java contains two main hierarchy trees:

1. The **Collection** hierarchy
2. The **Map** hierarchy

Map is part of collection framework, but it does not extent **Collection**. A Collection store individual elements, while a **Map** store key-value pair



Before java 21:

AL--->List---->Collection --> Iterable

After java 21:

AL--->List---->SequencedCollection-->Collection --> Iterable

Before 1.7

Interface ---> abstract

After 1.7

Interface

1. Abstract method
2. Default method
3. Static method

List Interface

A List is :

1. Ordered
2. Index-based
3. Duplicate allowed
4. Usually allow null

Realtime use of List

1. Youtube playlist
2. Shopping cart
3. Search history
4. Phone pe transaction

ArrayList

Is internally based on a dynamically resizable array.

```
List<String> playList= new ArrayList<>();  
private static final int DEFAULT_CAPACITY = 10;
```

```
playList.add("video: 1");  
playList.add("video: 2");  
playList.add("video: 3");  
playList.add("video: 1");
```

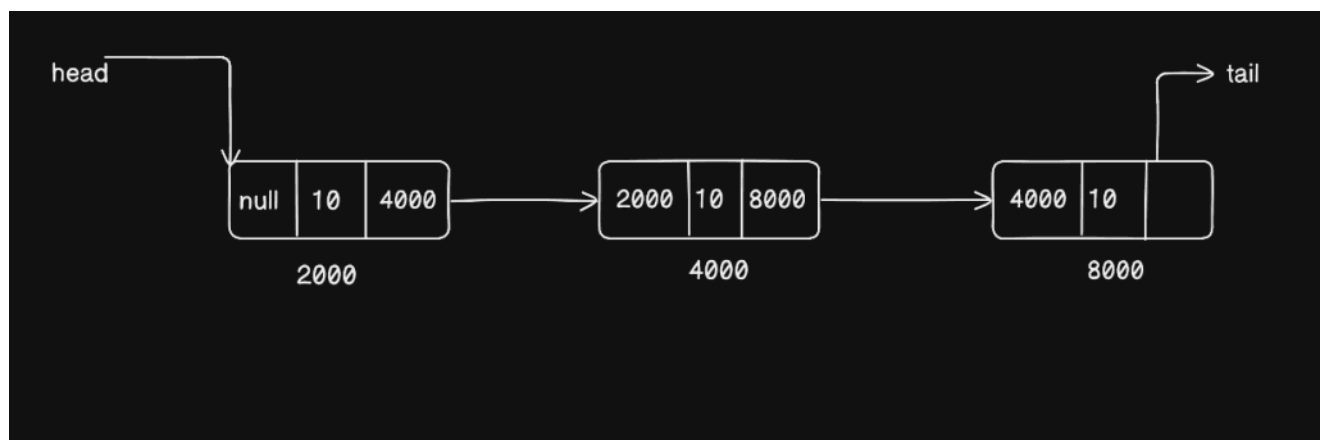
Why is ArrayList is fast for reading ?

The internal storage is an array, the address of an element can be calculate using index.

playList.get(1)

When to use ArrayList ?

1. Reading by index is frequent
2. Most insertion happen at the end
3. Iteration performance matter
4. Memory efficiency is important



Java's LinkedList is a doubly Linked List. Each node coceptually store

Prev ref

Data

Next ref

Why LinkedList is not best option in random access?

Because is used loop to find exact index

Why insertion at the end of LL is efficient ?

For insertion at the beginning or end, reference can be updated without shifting all elements.

Note: LinkedList insertion is always faster then ArrayList

Real Application:

1. Better CPU cache locality
2. Spotify search (LRU cache)

ArrayList Vs LinkedList

Operation	ArrayList	LinkedList
Access by index	$O(1)$	$O(n)$
Add at the end	$O(1)$	$O(1)$
Add at the beg	$O(n)$	$O(1)$
Seach by value	$O(n)$	$o(n)$

Vector

1. Is legacy resizable array.
2. Insertion order is preserved
3. Duplicate object are allowed
4. Null insertion is possible
5. Implements RandomAccess, Cloneable, java.io.Serializable
6. Initial capacity 10
7. Every method in Vector is synchronized and hence vector object is thread safe

Stack

1. It is child Class of Vector
2. It is specially designed for LIFO order

Methods:

1. Push()
2. Pop()
3. Peek()
4. Empty()

5. Search()

Set

A Set represents unique elements

```
Set<String> emails = new HashSet<>();
```

Realtime examples:

1. Unique email address
2. Unique course enrolments
3. User permissions
4. Tags assigned to a video
5. Unique coupon code
6. Removing duplicated values

HashSet

HashSet is backed internally by hash-based storage.

Conceptually, when an element is added

1. Java calculates its hash
2. The hash helps identify a bucket
3. Java checks whether an equal element already exists, if not element will be inserted.

Therefore, correct implementation of:

1. hashCode()
2. Equals()

Load factor:

Fill ratio 0.75 means after filling 75%

Performance wise

1. Add : O(1)
2. Remove : O(1)
3. Contains : O(1)

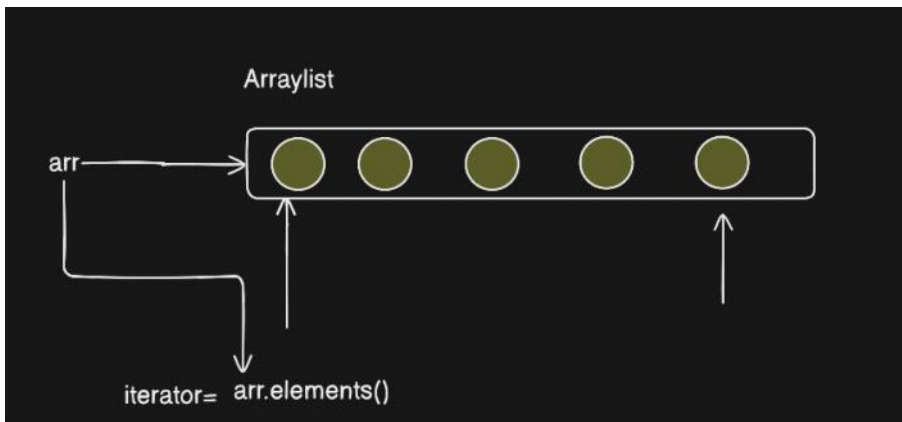
Cursors:

We have 3 types of cursors

1. Enumeration
2. Iterator
3. ListIterator

We want to get objects one by one from the collection then we should go for Cursors.

Enumeration:



Limitations of enumeration

1. It works for only legacy classes
2. By default it can only perform read operation, we can't perform remove operation

Iterator

It can perform read and remove operation

Methods:

`collectionObject.iterator()`
`hasNext()`
`next()`
`remove()`

Limitation:

1. It can move only in one direction(forward)
2. Only perform read & remove operation
3. To overcome above limitations we will go for `ListIterator`

ListIterator

1. `ListIterator` is child interface of `iterator`
2. It can move in both direction(forward + backward)
3. It can perform Read, replace & remove

`hasNext()`
`Next()`
`nextIndex()`

`hasPrevious()`
`Previous()`
`previousIndex()`

`Remove()`
`Set()`

LinkedHashSet

1. Unique elements
2. Usually insertion order preserve
3. Hash-based lookup

Realtime use case

Suppose search result contains duplicate product ids, but we want to preserve the first-seen order.

SortedSet

1. Child interface of Set if we want to store elements in some sorting order without duplicates we should go for SortedSet

TreeSet:

1. Underline data structure is balance tree
2. Duplicate object are not allowed
3. Insertion order not preserve
4. Null insertion possible (only once)
5. All object are inserted into some sorted order it may be default (natural sorting order) or customizer sorting order

Comparable And Comparator

Comparable:

It is present in java.lang package it contains only one method
compareTo(Object obj)

Example:

Obj1.compareTo(obj2) it will return 3 values

Obj1 : the object which is to be inserted

Obj2: the object which is already inserted

A.compareTo(Z)

Z.compareTo(A)

A,z

1. -ve if obj1 has come before obj2
2. +ve if obj1 has come after obj2
3. 0 if obj1 is equal to obj2

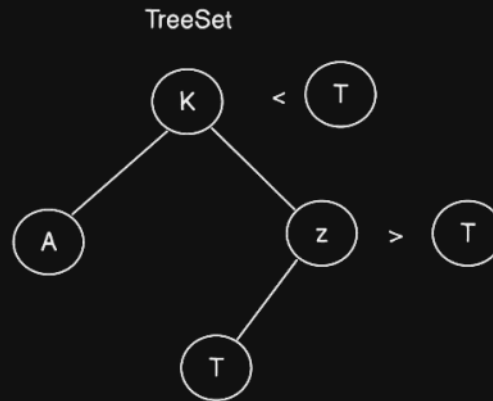
```

t.add("K");
t.add("Z");
t.add("A");
t.add("A");
t.add("T");

```

`z.compareTo(k) ----> +ve`

`A.compareTo(z) - ve`



Note: If we are depending on default natural sorting while adding objects onto TreeSet will go for **comparable**

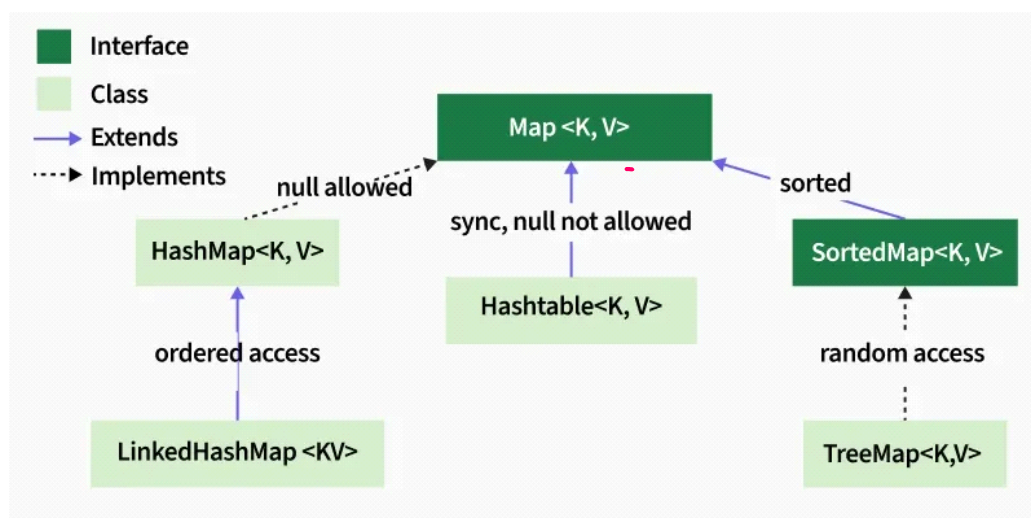
Comparator

Present in java.util package and it defines two methods `compare()` & `equals()`

`Int compare(obj1,obj2)`

`Equals(obje)`

Note: when we use comparator compulsory will give implementation of `compare()` only and we do not required to give implementation for `equals()` method because it is available to our class from Object class



Map Interface

A map store key-value pairs

Key	Value
101	Aman
102	Raman



A key must be unique, values may be duplicate

```
Map<Integer,String> students=new HashMap();
```

Methods:

1. Put()
2. Remove()
3. Size()
4. Clear()
5. keySet()
6. Values()
7. entrySet()

HashMap:

1. Underline data structure HashTable
2. Insertion order not preserve
3. Duplicate keys are not allowed
4. Null is allowed for key(only once)
5. Null allowed for values (any number of time)

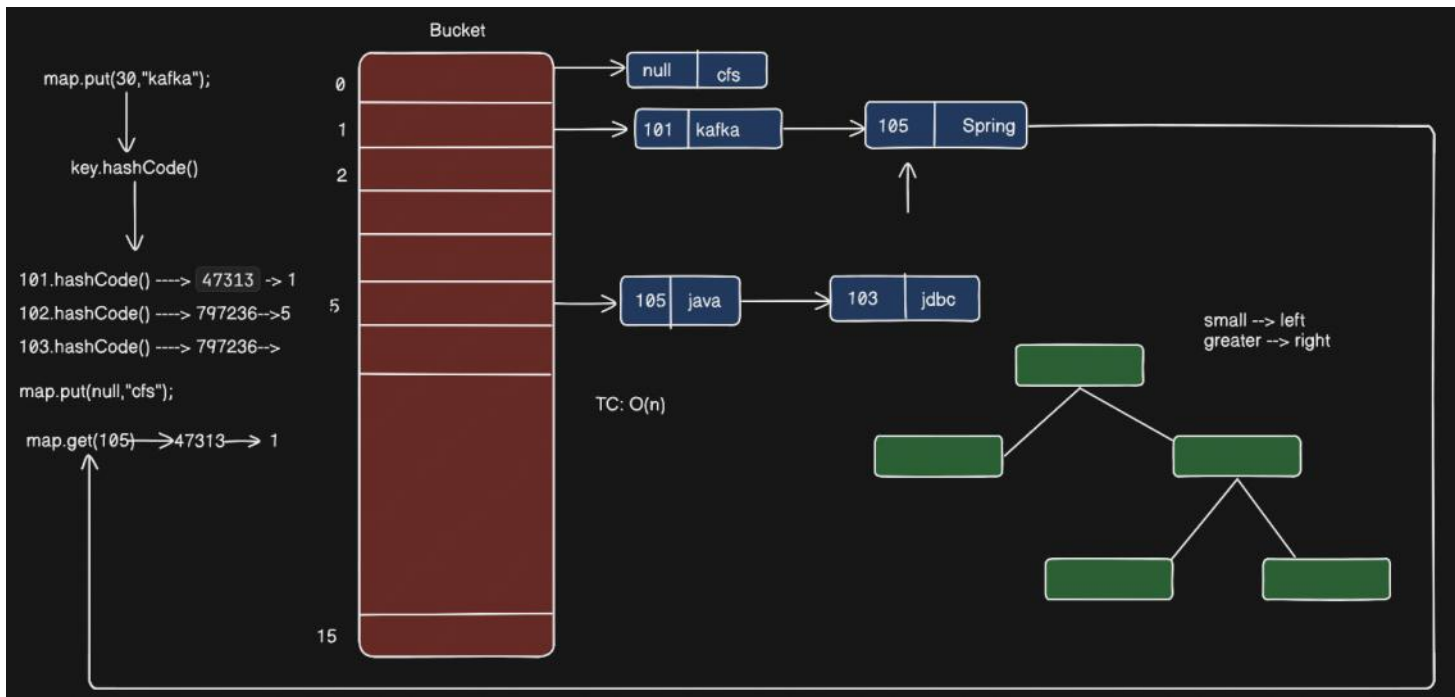
HashMap	HashTable
Not thread safe	Thread safe
Null is allowed for both key and values	Null is not allowed for keys and values
Introduced in 1.2v	It is legacy class comes in 1.0v

LinkedHashMap

It is child of HashMap

It is exactly same as HashMap but few changes are there

1. Underline data structure is LinkedList and Hashtable
2. Insertion order preserve



r

r